Step 3 — Retries, Then a Safe Fallback, When a Model Call Fails — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: real API calls time out, hit rate limits, and sometimes just fail. The retry part is already there: the OpenAI client under `ChatOpenAI` retries timeouts, connection errors, 429s and 5xx errors **2 times with backoff** (Doc04's error topic). Adding `.with_retry()` on top would stack a second retry layer — the mistake Doc02, Doc04 and Doc05 all warn about. What's missing is the part **after** the retries run out: today, the ticket crashes. This step gives every model call a safe backup. **If not:** one bad minute at OpenAI empties your whole queue into tracebacks.

After this step the project runs as before, and when the model can't be reached, the ticket gets a safe canned reply instead of a crash. Files not shown here don't change.

Changed first

Changed from Step 2: `main.py` no longer catches `RateLimitError` — the fallback now handles it inside the pipeline — and it prints whether the fallback was used. A bad key is still caught: it's not a temporary problem, and no fallback should hide it.

Changed from Step 2: `TicketRecord` gets its first new field, `used_fallback`, with a default.

Story — `schemas.py`: the record says whether the safe canned reply was sent. The default `False` means the Step 1 test that builds a record still works unchanged. **If not:** a fallback reply would look exactly like a real one in your data.

```
# src/langchainpro/schemas.py
from typing import Literal

from pydantic import BaseModel

class TicketCategory(BaseModel):
    # why: the classifier's answer as a checked object, not free text —
    # only these 3 values can ever come back (Doc04's Literal)
    category: Literal["billing", "technical", "general"]
    # why: one short sentence, logged, so a wrong route can be explained
    reason: str
    # why: False when the ticket could fit more than one category — a
```

```
    # guess the rest of the pipeline should know about
    sure: bool

class TicketRecord(BaseModel):
    # why: one checked record per ticket — what a ticketing system would
    # store, count and route on (a machine reads this, not a person)
    customer_id: str
    ticket: str
    category: str
    reply: str
    # how: every field added in a later step has a default, so code
    # (and tests) written in an earlier step keep working unchanged
    # new in Step 3
    used_fallback: bool = False
```

Story — **`main.py`** **:** only the errors a person must fix reach this loop now, and `print_record()` shows one more line. **If not:** a rate limit would still skip the ticket instead of answering it.

```
# src/langchainpro/main.py
import json
import sys

import openai

from langchainpro.config import load_config
from langchainpro.exceptions import MissingConfigError
from langchainpro.schemas import TicketRecord
from langchainpro.services.pipeline import handle_ticket
from langchainpro.utils.logger import get_logger

# how: a path from the repo root — run every command from there
SAMPLES_PATH = "data/sample/tickets.json"

def load_samples() -> list[dict]:
    # why: the same 4 tickets every run, shared with the tests (Doc02
    # taught reading JSON)
    with open(SAMPLES_PATH) as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def print_record(record: TicketRecord) -> None:
    print(f"  category:      {record.category}")
    # new in Step 3
    print(f"  used fallback: {record.used_fallback}")

def main() -> None:
```

```
    # why: config first — a missing OPENAI_API_KEY stops the app here
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    samples = load_samples()
    print(f"LangChainPro — {len(samples)} tickets in the queue.")

    for sample in samples:
        print()
        print(f"Ticket from {sample['customer_id']}: {sample['ticket']}")
        try:
            record = handle_ticket(sample["ticket"], sample["customer_id"])
        except openai.AuthenticationError:
            # changed in Step 3: no RateLimitError here any more
            # why: a bad key is permanent — no retry fixes it, so exit
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        print_record(record)

if __name__ == "__main__":
    main()
```

Back to this step · Hint 1 · Hint 2 · Solution

Solution — `.with_fallbacks()` on each chain, only for transient errors

Story — `chains/fallback.py`: the backups, in one file. `TRANSIENT_ERRORS` lists only failures that pass on their own. `with_canned_reply()` and `with_unsure_category()` wrap any chain with `.with_fallbacks([...], exceptions_to_handle=TRANSIENT_ERRORS)` (Doc05's LCEL topic); `RunnableLambda` turns a plain function into a step `.with_fallbacks()` accepts. The design decision is written at the top: a canned reply is fine even for a double charge, because it promises nothing and every fallback gets flagged. **If not:** a bug in your own code (a `KeyError`) would be silently hidden behind a polite canned reply.

```
# src/langchainpro/chains/fallback.py
import openai
from langchain_core.runnables import Runnable, RunnableLambda

from langchainpro.schemas import TicketCategory
from langchainpro.utils.logger import get_logger

# why: fall back only on failures that are nobody's fault and pass on
# their own — the same "transient" errors Doc04's table retries. A bad
# key or a bug in our own code must still fail loudly.
# how: the OpenAI client under ChatOpenAI already retried these 2 times
# with backoff (Doc04) — the fallback only runs when retries ran out
TRANSIENT_ERRORS = (
    # how: APITimeoutError is a kind of APIConnectionError
    openai.APIConnectionError,
    openai.RateLimitError,
    openai.InternalServerError,
)

# why: the design decision, on purpose — even for a double charge, this
# reply is safe to send: it promises nothing and admits nothing. Every
# fallback is also flagged for a human (Step 4), so nothing is hidden.
FALLBACK_REPLY = (
    "Thanks for contacting SupportCo. We've received your message, "
    "and a member of our team will reply within one business day."
)

def canned_reply(inputs: dict) -> str:
    logger = get_logger(__name__)
    # why: logged as a warning — a fallback should be visible in your
```

```
    # logs, not only in the customer's inbox
    logger.warning("Reply chain failed - sending the fallback reply")
    return FALLBACK_REPLY

def unsure_category(inputs: dict) -> TicketCategory:
    logger = get_logger(__name__)
    logger.warning("Classifier failed - using general, marked unsure")
    return TicketCategory(
        category="general",
        reason="The classifier was unavailable.",
        sure=False,
    )

def with_canned_reply(chain: Runnable) -> Runnable:
    # how: .with_fallbacks() runs the backup only when the chain raises
    # one of these errors (Doc05); RunnableLambda turns our plain
    # function into a step that .with_fallbacks() accepts
    backup = RunnableLambda(canned_reply)
    return chain.with_fallbacks(
        [backup], exceptions_to_handle=TRANSIENT_ERRORS
    )

def with_unsure_category(chain: Runnable) -> Runnable:
    backup = RunnableLambda(unsure_category)
    return chain.with_fallbacks(
        [backup], exceptions_to_handle=TRANSIENT_ERRORS
    )
```

Changed from Step 2: the classifier chain is wrapped with its backup.

Story — `chains/classifier.py`: if the classifier can't be reached, the ticket becomes "general, not sure" — and still gets a reply. **If not:** a classifier outage would stop every ticket before any reply chain even ran.

```
# src/langchainpro/chains/classifier.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from pydantic import ValidationError

# new in Step 3
from langchainpro.chains.fallback import with_unsure_category
from langchainpro.models.llm import get_model
from langchainpro.schemas import TicketCategory
from langchainpro.utils.logger import get_logger
from langchainpro.utils.prompts import load_prompt
```

```
def build_classifier_chain() -> Runnable:
    # why: prompt | structured model — Doc05's "shape during" method, so
    # the answer arrives as a checked TicketCategory, not text to parse
    prompt = ChatPromptTemplate.from_template(load_prompt("classifier"))
    model = get_model().with_structured_output(TicketCategory)
    chain = prompt | model
    # why: if the model is down even after the client's retries, the
    # ticket still gets a category (general, marked unsure)
    # changed in Step 3: wrapped
    return with_unsure_category(chain)

def classify_ticket(ticket: str) -> TicketCategory:
    logger = get_logger(__name__)
    chain = build_classifier_chain()
    try:
        result = chain.invoke({"ticket": ticket})
    except ValidationError:
        # why: an answer that doesn't fit the shape must not crash the
        # pipeline — the safe default is "general, and not sure"
        logger.warning("Classifier answer did not fit TicketCategory")
        result = TicketCategory(
            category="general",
            reason="The classifier's answer could not be read.",
            sure=False,
        )
    # why: every routing decision is logged with its reason (Project 1)
    logger.info(
        f"category={result.category} sure={result.sure} "
        f"reason={result.reason}"
    )
    return result
```

Changed from Step 2: every reply chain is wrapped with the canned reply.

Story — `chains/replies.py` **:** the wrapper goes around the **whole** chain, lookups included, so any transient failure in it falls back. **If not:** you'd wrap three chains three times.

```
# src/langchainpro/chains/replies.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable

# new in Step 3
from langchainpro.chains.fallback import with_canned_reply
from langchainpro.chains.lookups import build_lookup_step
from langchainpro.models.llm import get_model
```

```
from langchainpro.utils.prompts import load_prompt

def build_reply_chain(category: str) -> Runnable:
    # why: this is the branch — one reply chain per category, each with
    # its own prompt file (billing.txt, technical.txt, general.txt)
    # how: the category name IS the prompt file's name
    prompt = ChatPromptTemplate.from_template(load_prompt(category))
    chain = prompt | get_model() | StrOutputParser()

    if category == "general":
        # why: a general question needs no account data — looking it up
        # anyway would only add a second of waiting
        full_chain = chain
    else:
        # how: the lookup step's dict (ticket, account, orders) fills
        # the prompt's {ticket}, {account} and {orders}
        full_chain = build_lookup_step() | chain

    # why: if the model is down even after retries, send a safe reply
    # changed in Step 3: wrapped
    return with_canned_reply(full_chain)
```

Changed from Step 2: the record says whether the fallback ran.

Story — `services/pipeline.py`: the canned reply is always the exact same text, so comparing to `FALLBACK_REPLY` tells you it was used. **If not:** a fallback would be invisible in your data — Step 4 needs this flag.

```
# src/langchainpro/services/pipeline.py
from langchainpro.chains.classifier import classify_ticket
# new in Step 3
from langchainpro.chains.fallback import FALLBACK_REPLY
from langchainpro.chains.replies import build_reply_chain
from langchainpro.schemas import TicketRecord

def handle_ticket(ticket: str, customer_id: str) -> TicketRecord:
    # how: classify -> pick the reply chain -> get the reply -> build
    # the record
    category = classify_ticket(ticket)
    chain = build_reply_chain(category.category)
    # how: every reply chain gets the same input dict; a prompt simply
    # ignores keys it doesn't use (Doc05's template edge cases)
    inputs = {"ticket": ticket, "customer_id": customer_id}
    reply = chain.invoke(inputs)
    print(f"Reply: {reply}")
```

```
    # why: the fallback always returns exactly FALLBACK_REPLY, so an
    # equal reply means the fallback ran
    # new in Step 3
    used_fallback = reply == FALLBACK_REPLY

    return TicketRecord(
        customer_id=customer_id,
        ticket=ticket,
        category=category.category,
        reply=reply,
        # new in Step 3
        used_fallback=used_fallback,
    )
```

Story — `tests/unit/test_fallback.py`: a fallback you can't prove fires — and can't prove stays quiet — isn't tested. Case 1 points a real `ChatOpenAI` at a port where nothing listens: a real, forced outage with no key and no cost. Case 2 proves a working chain never touches the fallback. Case 3 proves a bug is **not** hidden. **If not:** you'd find out the fallback doesn't work during a real outage.

```
# tests/unit/test_fallback.py
# Proves the fallback runs when the model can't be reached, and only
# then. No API key needed. Run from the repo root:
#   python tests/unit/test_fallback.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda
from langchain_openai import ChatOpenAI

from langchainpro.chains.fallback import FALLBACK_REPLY, with_canned_reply

def build_broken_chain():
    # how: nothing listens on port 9 of your own machine, so every call
    # fails at once with a connection error — a real, forced outage,
    # with no API key and no cost
    # why: max_retries=0 skips the client's 2 retries, so it's instant
    model = ChatOpenAI(
        model="gpt-4o-mini",
        api_key="sk-test",
        base_url="http://127.0.0.1:9",
        max_retries=0,
    )
    prompt = ChatPromptTemplate.from_template("Reply to: {ticket}")
    return prompt | model | StrOutputParser()

def working_step(inputs: dict) -> str:
    return "A real reply."
```

```
def buggy_step(inputs: dict) -> str:
    # why: a bug in our own code, not an outage
    return inputs["no_such_key"]

def test_outage_uses_fallback() -> None:
    chain = with_canned_reply(build_broken_chain())
    reply = chain.invoke({"ticket": "Where is my invoice?"})
    print(f"case 1 (outage)           -> fallback={reply == FALLBACK_REPLY}")

def test_working_chain_skips_fallback() -> None:
    chain = with_canned_reply(RunnableLambda(working_step))
    reply = chain.invoke({"ticket": "Hi"})
    print(f"case 2 (working)          -> {reply}")

def test_bug_is_not_hidden() -> None:
    # why: a KeyError is not a transient error — it must still raise
    chain = with_canned_reply(RunnableLambda(buggy_step))
    try:
        chain.invoke({"ticket": "Hi"})
        print("case 3 (bug)              -> hidden (wrong!)")
    except KeyError:
        print("case 3 (bug)              -> KeyError still raised")

if __name__ == "__main__":
    test_outage_uses_fallback()
    test_working_chain_skips_fallback()
    test_bug_is_not_hidden()
```

Run it:

```
python tests/unit/test_fallback.py
python -m langchainpro.main
```

`test_fallback.py` prints (after one warning log line):

```
case 1 (outage)           -> fallback=True
case 2 (working)          -> A real reply.
case 3 (bug)              -> KeyError still raised
```